

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ  
БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем  
Факультет автоматики и вычислительной техники  
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №1  
по дисциплине  
«Алгоритмы и структуры данных»

Выполнил студент гр. ИВТб-1303-06-00 \_\_\_\_\_/Гортоломей И.К./  
Проверил заведующий кафедрой ЭВМ \_\_\_\_\_/Долженкова М.Л./

Киров  
2026

## **Цель работы:**

формирование навыков алгоритмического мышления посредством решения набора задач с predetermined характеристиками и уровнем сложности.

## **Номера заданий:**

1. <https://codeforces.com/contest/1760/problem/C>
2. <https://codeforces.com/contest/1807/problem/D>
3. <https://codeforces.com/contest/2123/problem/C>
4. <https://codeforces.com/contest/522/problem/B>
5. <https://codeforces.com/contest/1598/problem/C>

## **Решение заданий**

## Задание 1:

Листинг 1: Исходный код программы (C)

```
1 // https://codeforces.com/contest/1760/problem/C
2 #include <stdio.h>
3
4 int main() {
5     int i;
6     scanf("%d", &i);
7     for (; i > 0; i--) {
8         int j, n; scanf("%d", &j); n = j;
9         int a[j], fm = 0, fmi = -1, sm = 0;
10        for (j = 0; j < n; j++) {
11            scanf("%d", &a[j]);
12            if (a[j] > fm) {
13                sm = fm;
14                fm = a[j];
15                fmi = j;
16            } else if (a[j] > sm) {
17                sm = a[j];
18            }
19        }
20        for (j = 0; j < n; j++)
21            printf("%d ", a[j] - (j == fmi ? sm : fm));
22        printf("\n");
23    }
24    return 0;
25 }
```

Суть данного задания - найти между  $a_i$  элементом и максимальным разницу. Так как  $a_i$  элемент сам может быть максимальным, то надо найти следующий после него максимальный элемент. Максимальный и следующий элемент можно найти в цикле при вводе массива чисел, А в следующем цикле можно вывести разницу.

## Задание 2:

Листинг 2: Исходный код программы (C)

```
1 // https://codeforces.com/contest/1807/problem/D
2 #include <stdio.h>
3
4 int main() {
5     int t; scanf("%d", &t);
6     for (; t > 0; t--) {
7         int n, q, x;
8         scanf("%d %d", &n, &q);
9         long long a[n+1];
10        a[0] = 0;
11        for (int i = 0; i < n; i++) {
12            scanf("%d", &x);
13            a[i+1] = a[i] + x;
14        }
15        for (; q > 0; q--) {
16            int l, r, k;
17            scanf("%d %d %d", &l, &r, &k);
18            int sum = a[n] - a[r] + a[l-1] + k*(r-l+1);
19            printf(sum % 2 != 0 ? "YES\n" : "NO\n");
20        }
21    }
22    return 0;
23 }
```

Суть данного задания - заменить в отрезке массива числа на  $k$  и проверить, является ли сумма чисел массива нечётной. Для оптимального решения можно заранее вычислить суммы, так что  $a_{i+1} = a_i + x$  а далее из суммы вычитаем значения на промежутке и добавляем  $r - l + 1$  раз число  $k$

### Задание 3:

Листинг 3: Исходный код программы (C)

```
1 // https://codeforces.com/contest/2123/problem/C
2 #include <stdio.h>
3
4 int main() {
5     int t; scanf("%d", &t);
6     for (; t > 0; t--) {
7         int n; scanf("%d", &n);
8         int a[n]; scanf("%d", &a[0]);
9         int pm[n], sm[n];
10        pm[0] = a[0];
11        for (int i = 1; i < n; i++) {
12            scanf("%d", &a[i]);
13            pm[i] = (a[i] < pm[i-1] ? a[i] : pm[i-1]);
14        }
15        sm[n-1] = a[n-1];
16        for (int i = n-2; i >= 0; i--)
17            sm[i] = (a[i] > sm[i+1] ? a[i] : sm[i+1]);
18        printf("1");
19        for (int i = 1; i <= n-2; i++)
20            printf(a[i] < pm[i-1] || a[i] > sm[i+1] ? "1" : "0");
21        printf("1\n");
22    }
23    return 0;
24 }
```

Суть данного задания - проверить каждое  $a_i$  есть ли для него такая комбинация замены префикса(мин)/суффикса(макс), чтобы оно заполнило весь массив. Для решения этого задания вводятся массивы  $pm$  и  $sm$  в которых хранятся минимальные числа слева от  $a_i$  и максимальные справа от  $a_i$  соответственно. Если  $a_i$  меньше чем числа слева или больше чем числа справа - то выводим 1, иначе 0.

## Задание 4:

Листинг 4: Исходный код программы (C)

```
1 // https://codeforces.com/contest/522/problem/B
2 #include <stdio.h>
3
4 int main() {
5     int n; scanf("%d", &n);
6     int sum = 0, w[n], h[n];
7     int fm = 0, fmi = -1, sm = 0;
8     for (int i = 0; i < n; i++) {
9         scanf("%d", &w[i]);
10        scanf("%d", &h[i]);
11        sum += w[i];
12        if (h[i] > fm) {
13            sm = fm;
14            fm = h[i];
15            fmi = i;
16        } else if (h[i] > sm) {
17            sm = h[i];
18        }
19    }
20    for (int i = 0; i < n; i++)
21        printf("%d ", (sum - w[i])*(i == fmi ? sm : fm));
22    return 0;
23 }
```

Суть данного задания - вычислить размер фотографии  $w \cdot h$ , где  $w$  - сумма  $w_i$ , а  $h$  - максимальное  $h_i$  для каждого друга, так как один из друзей фотографирует остальных. Для решения задачи суммируем  $w$ , находим максимальный и следующий после него  $h$  и вычисляем размер, вычитая из суммы  $w_i$  и умножаясь на максимальное (или на следующее максимальное).

## Задание 5:

### Листинг 5: Исходный код программы (Python)

```
1 # https://codeforces.com/contest/1598/problem/C
2 from sys import stdin
3 from collections import defaultdict
4
5 t = int(stdin.readline())
6 for _ in range(t):
7     n = int(stdin.readline())
8     a = list(map(int, stdin.readline().split()))
9     s = sum(a)
10
11     if (2 * s) % n != 0:
12         print(0); continue
13
14     target = (2 * s) // n
15     count = defaultdict(int)
16     ans = 0
17
18     for x in a:
19         need = target - x
20         ans += count[need]
21         count[x] += 1
22     print(ans)
```

Суть данного задания - найти кол-во пар элементов, которые можно удалить из массива чтобы среднее арифметическое не поменялось. Для реализации алгоритма находим целевую сумму чисел, если сумма вещественная - то из входных данных мы не найдём подходящей пары. В цикле проходим по элементам и находим элемент, который нам нужен  $need = target - x$ , добавляем к счётчику число из словаря  $ans + count[need]$ , и увеличиваем значение текущего эл. в словаре  $count[x] + 1$

## Выводы

Решение данных задач позволяет разобраться в простой оптимизации алгоритма, чтобы уложиться в жёсткий лимит по времени. Если пытаться решать эти задачи в лоб, как цикл в цикле или перебором, то ничего не получится. Для решения нужно заранее вычислять значения, чтобы приблизиться к  $O(n \cdot \log(n))$  или даже  $O(n)$ . Основным способом оптимизации - это "аккумулирующие" массивы, которые могут хранить суммы, произведения, макс/мин значения и так далее. Например, во второй задаче используется массив для вычисления сумм, чтобы каждый раз не пересчитывать сумму массива и не изменять массив. А в третьей задаче массивы используются для нахождения минимального/максимального значения. В первой и четвертой задаче заранее вычисляется максимум и следующее число максимума, для того чтобы сравнить индекс числа и индекс максимального числа и в цикле решить с чем сравнивать. В пятой задаче проще всего на Python решить задачу, так как проще использовать словари - они используются для подсчёта пар чисел. Вообще, для решения задач нужно не просто перебирать значения или "в лоб" решать, а надо оптимизировать кол-во повторений/проход в цикле, а также следует подбирать подходящие инструменты индивидуально для каждой задачи.